# DHAP

## *Product Vision & Technical Blueprint*

NH Prince Pradhan

22 October 2026

**Make the steps clear.**

*A local-first screen recorder and documentation studio.*

**Working brand:** Dhap (ধাপ)
**Document status:** Product specification draft v1.0
**Research snapshot:** 22 October 2026
**Purpose:** A source of truth for product design, engineering, testing, release automation, and launch.

# Contents


- Executive summary
- 1. Product definition
- 2. Naming and brand decision
- 3. Market landscape and differentiation
- 4. Product principles and commitments
- 5. Scope and release strategy
- 6. Core user workflows
- 7. Functional requirements
- 8. Information architecture and UI/UX
- 9. Recommended technology and architecture
- 10. Project format and data model
- 11. Privacy, security, and responsible AI
- 12. CI/CD and release automation
- 13. Testing and acceptance criteria
- 14. Roadmap: 12-week solo execution plan
- 15. Public release, website, and community
- 16. Sustainability and commercial strategy
- 17. Licensing and upstream responsibilities
- 18. Risks and mitigations
- 19. Decision log and unresolved items
- 20. Definition of done for v1.0
- 21. Research references

# Executive summary

Dhap turns the work a person performs on screen into two useful things from one source project: a polished recording and a clear, editable guide. Users can record the screen or import existing media, select or generate meaningful steps, annotate screenshots, correct the wording, protect sensitive information, and export video plus documentation without needing an account or an online service.

The core product promise is **record once, edit once, publish in the formats people actually need**. It is not "an AI that writes documentation" and not merely a video editor with a different skin. It is a unified workspace where video, timeline events, screenshots, written instructions, annotations, and exports share one editable source of truth.

The recommended foundation is to evaluate and then deeply adapt the MIT-licensed OpenScreen project, retaining its existing cross-platform capture and rendering capabilities where practical. OpenScreen's documentation describes native capture paths for macOS, Windows, and Linux, a Rust GPU rendering core, transcription, an optional LLM editing layer, CLI/MCP automation, and MP4/GIF export. It is a strong starting point, but its CLI and project format are actively evolving, so Dhap must isolate integration points and own a stable, versioned guide-project format. [1][2]

The full ambition is broad, but a solo developer cannot responsibly ship recording, a full video editor, browser capture, every export, AI orchestration, a web editor, batch processing, plugins, team collaboration, and mature cross-platform automation in three months. This blueprint therefore distinguishes a polished 12-week MVP from later releases. The first release should deliver a trustworthy workflow on desktop; a browser-accessible viewer and exported interactive guides are in scope, while a full web editor and multi-user collaboration are not.

**Product principles:** no account required; no paywall on core capabilities; local files remain the user's property; AI is optional; all AI suggestions are editable and reviewable; sensitive data should never be silently uploaded; exports must be reproducible and reliable; and updates must never forcibly close the user's work.

# 1. Product definition

## 1.1 One-sentence definition

Dhap is a polished, open-source desktop studio that records or imports screen activity and transforms it into editable video, screenshots, and step-by-step documentation from the same project.

## 1.2 The problem

People commonly separate the process into several tools: one tool records a video, another extracts screenshots, a document editor holds the instructions, an image editor adds arrows and blur, and a final tool creates a PDF or website. When the interface changes, the video and guide drift apart. Manual step-writing is repetitive, redaction is easy to forget, and output formats require duplicated work.

Dhap should reduce this fragmentation. Its core value is not merely automation; it is **one source of truth with several useful outputs**, with a human editor controlling the final result.

## 1.3 Product goals

1. Make it easy to go from a real task to a high-quality, editable guide.

2. Preserve the creative polish of a modern desktop product without turning it into a cluttered professional video-editing suite.
3. Work offline for recording, editing, project saving, and core exports.
4. Support Windows, macOS, and Linux through a maintained upstream foundation rather than reimplementing platform capture.
5. Keep advanced capabilities available without usage caps or account gates.
6. Make every release repeatable: CI, tests, packaging, checksums, release notes, and deployment should be automated.
7. Build an open-source community around reliable workflows, templates, adapters, and export themes.

### 1.4 Non-goals for the first release

- Rebuilding a general-purpose video editor from scratch.
- Building a cloud-first SaaS or requiring an account to unlock core features.
- Recording every keystroke or storing typed password/form contents.
- Automatically publishing a guide without review.
- Supporting full collaborative multi-user editing in the first three months.
- Reimplementing all of OpenScreen's media pipeline in a different framework.
- Claiming imported video can reveal exact clicks or DOM actions when those events were never recorded.

### 1.5 Primary users and launch wedge

The long-term product may serve anyone who creates step-by-step guides. The first audience must be narrower so the project can validate value quickly.

**Recommended launch wedge:** open-source maintainers, developers, technical educators, and IT/support people who frequently explain how to install, configure, reproduce, or operate software. They already understand Markdown and GitHub, produce guides repeatedly, and can give specific feedback.

Secondary audiences: teachers and students, product teams, internal operations, QA teams, and tutorial creators. Dhap should not require any particular audience's terminology to create a useful guide.

### 1.6 Jobs to be done

- "When I finish a workflow, help me create a guide without repeating the task for screenshots and prose."
- "When I record a tutorial, let me deliver both a video and a written version from the same source."
- "When I share documentation, help me catch secrets and private information before it leaves my machine."
- "When a UI changes, let me update only the affected step and regenerate the outputs."
- "When I use GitHub or another docs site, give me clean files I own, not a locked hosted page."

## 2. Naming and brand decision

### 2.1 Recommended name: Dhap

**Dhap** is the strongest current candidate. In Bengali, ধাপ **(dhap)** means a step of a staircase and, by extension, a stage, level, or phase in a process. "ধাপে ধাপে" means step by step. That gives the name an authentic linguistic origin and a direct, memorable relationship to the product's output without describing every feature literally. The word is short, visually strong, and consistent with the kind of natural, meaningful name that made Khata distinctive. [3]

**Recommended written brand:** Dhap
**Pronunciation guide for the website:** dhap (one syllable; Bengali ধাপ)
**Tagline:** Make the steps clear.
**Supporting line:** Record once. Create the video and guide together.

The name is a *provisional brand recommendation*, not legal clearance. Dhap is also an ordinary Bengali word, which can limit distinctiveness in some contexts. Before publishing the brand, verify domain registration and trademark availability in the regions and product classes that matter. The browser research completed for this specification did not establish an authoritative domain-availability result or a complete trademark clearance for Dhap. Do not claim that dhap.com, dhap.app, dhap.dev, dhap.tools, or a GitHub organisation is available until checked live.

## 2.2 Preliminary naming screen

| Candidate | Why it fits | Public collision found | Recommendation |
|---|---|---|---|
| **Dhap** | Bengali meaning: step / stage; short and directly related to guided workflows | No dominant exact-match software product surfaced in the preliminary general web/GitHub searches; this is not a clearance result | **First choice, subject to formal checks** |
| Noksha | Bengali meaning associated with design/pattern/plan; creative character | A design/art-generation app named Noksha has been listed for iOS, although the listing appears archived | Backup only; assess collision and category confusion [4] |
| Sopan | Means steps/staircase in Sanskrit-derived usage; very apt metaphor | Active education/workflow SaaS brand using SOPAN | Avoid due to close product-metaphor and SaaS overlap [5] |
| Kadam | South Asian word/name associated with a step/footstep | Multiple current fitness applications use Kadam | Avoid; high app-store search and naming noise [6] |
| Lekha | Means writing/account/record in South Asian languages | Current business-management software and a separate ledger app use Lekha | Avoid; relevant software-category overlap [7] |
| Poth | Bengali for path/way | An existing ebook/content-reader app called Poth appears in app stores | Avoid; discoverability and category confusion [8] |
| Chitro | Bengali word associated with picture/image | A video editor named Chitro already exists | Avoid; strong video-category overlap [9] |

This is a preliminary public-market screen, not a complete trademark, company-name, social-handle, or domain audit. Name research must include similar spellings and pronunciations, not just exact matches.

## 2.3 Required pre-launch checks

1. Check the intended domains via an ICANN/RDAP lookup and a registrar checkout. Check .com, .app, .dev, .tools, .video, and the preferred local/brand extension. Confirm renewal price and registry restrictions, not only the first-year price. [10]
2. Search WIPO's Global Brand Database and the relevant national trademark registers, including Bangladesh and the main planned markets. Search Dhap, phonetic variants, and Bengali script ধাপ, especially for downloadable software, SaaS, media tools, and educational publishing. WIPO itself recommends checking national/regional registers in addition to its global database. [11]
3. Search GitHub organisations/repos, package registries, browser extension stores, App Store/Google Play, social handles, and software directories.
4. If the name passes, reserve the domains and accounts before public launch. Obtain professional trademark advice before significant marketing spend or commercial expansion.

## 2.4 Brand character

Dhap should feel **crafted, warm, confident, and useful**—human-made rather than machine-generated. It should not imitate Khata's paper notebook identity, though both may share the owner's preference for thoughtful, expressive design.

- Wordmark: bespoke letterforms or a carefully adjusted typeface; no generic AI sparkle.
- Icon: a small ascending sequence of three or four offset steps, or one step-shaped frame mark. Avoid a literal camera icon as the whole identity.
- Illustration system: human-drawn arrows, circles, underlines, small numbered markers, and visual examples that feel intentional rather than childish.
- Interface: precise layout and typography with a few warm creative details. The editor must remain calm during complex tasks.
- Suggested palette: ink/charcoal for structure, warm cream in light mode, deep slate in dark mode, a botanical green for progress, and a restrained vermilion/coral for recording and warnings. Colors are initial art direction, to be validated for contrast and not locked before UI prototypes.
- Motion: small, purposeful transitions; no decorative animation that delays work. Respect reduced-motion settings.
- Photography/marketing: show real screenshots, real guides, and before/after workflows. Avoid overused AI-generated abstract backgrounds.

# 3. Market landscape and differentiation

## 3.1 Evidence from adjacent projects

- **OpenScreen** is a mature, actively maintained MIT-licensed screen recorder and editor. Its current documentation, updated 17 October 2026, describes native capture paths across macOS, Windows, and Linux, GPU rendering through a Rust core, transcription, optional user-configured LLM editing, CLI/MCP automation, and MP4/GIF exports. It therefore provides a better foundation than a new capture engine. Its project format and CLI may change between releases, so Dhap needs a stable adapter boundary. [1][2]
- **Mimik** is an MIT-licensed browser extension that records browser interactions into annotated, editable step-by-step guides, with optional AI features, replay, and multi-format export. It validates demand for automatic guide creation inside the browser, but its core capture model is browser-focused. [12]
- **OpenSteps** is an MIT-licensed, Windows-first desktop workflow recorder. Its public README describes it as an early beta and calls out local capture, editable steps, annotated screenshots, and Markdown/HTML export. [13]

These projects show that recording and guide generation are both real product categories. They also mean Dhap must not pitch itself as the first tool that turns actions into steps. Differentiation must be visible in its combined media workflow and execution quality.

## 3.2 Dhap's intended difference

**One editable project produces the video and the documentation.** Dhap's central object is not just a recording or a list of clicks; it is a versioned project that relates source media, timestamps, steps, screenshots, annotations, narration/transcript, redactions, and export settings.

Dhap should differentiate on:

1. **Unified outputs:** video/GIF plus Markdown, styled HTML, PDF, and optionally an interactive guide from one reviewed project.
2. **Cross-platform desktop workflow:** import any common screen recording and create guides even where browser DOM capture is unavailable.
3. **Source-linked edits:** each guide step points to its source timestamp and frame, so a user can locate and revise the original moment.
4. **Safety-first review:** redactions, preview, and a pre-export checklist before any output is published or sent to an AI provider.
5. **Open artifacts:** Markdown, HTML, images, and a documented project bundle are portable. No proprietary hosted page is required.
6. **Careful craft:** an editor that feels fast and beautiful without sacrificing keyboard access, accessibility, or reliability.
7. **Agent automation as an interface, not a dependency:** CLI and MCP can be added around stable project operations while the full desktop workflow remains usable without an agent.

### 3.3 Competitive position

Do not attempt to win by matching every feature in every competing product. Win at the complete workflow: capture/import → step detection → human edit → safety review → multiple outputs. Keep the first-run experience simple and make advanced capabilities progressively discoverable.

## 4. Product principles and commitments

1. **Free core, no arbitrary caps.** No watermarks, guide-count limits, export limits, feature tiers for core local editing, or account wall.
2. **Local-first.** Recordings, projects, screenshots, transcripts, and exports stay on the device unless the user deliberately shares them.
3. **AI optional.** The core workflow works if no model is installed and no key is configured. No “AI quota” can block ordinary editing or exports.
4. **Human approval.** Auto-generated steps are drafts. The user can inspect, edit, delete, reorder, or reject suggestions.
5. **No forced interruptions.** Updates may download in the background if enabled, but never close an active recording, discard a project, or restart the app without consent.
6. **Safety over false confidence.** Redaction is explicit and testable. The app must not claim that a video is safe to share merely because a detector found no secrets.
7. **Portable content.** Exported outputs should remain usable outside Dhap. The source project should be documented and versioned.
8. **Honest automation.** Label estimates as suggestions and preserve links to source timestamps. A video-only import cannot reliably reveal every click, keystroke, or page navigation.
9. **Accessibility is foundational.** Keyboard navigation, clear focus, contrast, reduced motion, and screen-reader labels are release requirements.
10. **Upstream respect.** Preserve notices, make upstream attribution clear, and avoid implying endorsement by OpenScreen or its maintainers.

## 5. Scope and release strategy

### 5.1 Release levels

| Level | Objective | Scope |
|---|---|---|
| Prototype | Prove the workflow | Import a video, mark 5–10 steps manually, attach screenshots, export a guide. Test with real users before architecture hardens. |
| 12-week v1.0 | Polished, useful public release | Desktop cross-platform foundation; screen recording and video import; timeline-linked steps; scene/step suggestions; annotations/redaction; Markdown, HTML, PDF; preserve video/GIF export; local projects; optional AI; automated builds and releases. |
| v1.x | Expand capture and creator features | Better OCR/step detection, templates, batch processing, richer interactive guides, additional platform package formats, browser extension beta, CLI/MCP stable commands. |
| v2.x | Larger workflow ecosystem | Browser capture integration, full interactive walkthroughs, plugin contract, optional self-hosted/team collaboration, hosted publishing as an opt-in service, version comparison, broader localization. |

### 5.2 What is in the 12-week MVP

**P0 — required to ship** - Native desktop application based on evaluated OpenScreen architecture. - Record a screen/window using inherited OS-specific capture; preserve essential recorder reliability. - Import common

video formats supported by the selected media pipeline. - Create a new project; autosave; reopen project; recover from a crash where possible. - Generate candidate steps from scene changes and selected frame intervals; support manual step creation/editing/reordering. - Select a source frame for each step and store its source timestamp. - Add numbered markers, arrows, highlights, crop, blur/pixelate, and opaque redaction masks. - Edit step title, instructions, expected result, and optional notes. - Export Markdown plus related assets; export a styled self-contained HTML guide; export a printable PDF. - Preserve existing MP4/GIF export capability from the upstream foundation, subject to a proof-of-concept and integration tests. - Store the source project in a documented, versioned local bundle. - Light and dark themes, keyboard basics, accessible focus and readable error states. - CI, core automated tests, cross-platform build/package pipeline, checksums, generated release notes, and a working project website.

**P1 — include if the P0 workflow is stable** - Optional transcription using an existing local/on-device engine already present in the selected foundation. - AI-generated step text through a provider adapter after explicit opt-in. - Reusable guide/export templates. - Project snapshots/version history and export presets. - Portable Linux build and a Windows portable archive.

**Not a 12-week promise** - Full web editor with native-equivalent recording. - Complete browser extension across Chrome/Firefox/other browsers. - Multi-user collaboration, shared workspace servers, hosted accounts, and a sync engine. - Perfect OCR on every language and application. - One-click documentation for every arbitrary recording with no review. - Universal plugin ecosystem and stable public SDK.

### 5.3 Product platform policy

- **Desktop is the editing/recording product.** Target Windows, macOS, and Linux through the upstream capture/rendering foundation.
- **Web v1 is an output format and viewer.** A self-contained HTML guide and interactive published page can run in a browser without a login. The full recording/editor UI should not be ported to the web in v1.
- **Browser extension comes later.** It can record DOM-level context that a desktop video alone cannot reliably infer, using a separate adapter into the same project model.
- **Portable build:** ship at least a portable Windows archive and portable Linux AppImage/tar artifact where the packaging foundation allows; keep installation path and configuration documented.

## 6. Core user workflows

### 6.1 First run and onboarding

1. The user opens Dhap; no account or cloud setup is required.
2. Welcome screen offers three clear actions: **Record a task**, **Import a recording**, or **Create a guide from screenshots**.
3. A small sample project can be opened to demonstrate the final result before any permission request.
4. If recording is chosen, show a plain-language permission explanation immediately before the OS prompt and explain how to change it later.
5. Ask only essential first-run preferences: theme and default export directory. AI/provider setup is skipped by default.
6. The user can finish a basic guide without viewing a tour or connecting a service.

### 6.2 Record a task

1. Choose full screen, window, or a supported region/source. Select microphone/system audio/webcam only if available and wanted.
2. Show a visible recording indicator and simple pause/resume/finish controls.
3. Optional “mark a step” hotkey inserts a timestamp marker during recording. This is more reliable than trying to infer every action later.
4. Optional click markers may be recorded only with an explicit setting and platform permission. Typed values and passwords must not be recorded by default.

5. Stop recording. Preserve the raw source; do not destructively overwrite it.
6. Open the review workspace with the video timeline and suggested key moments.

## 6.3 Import an existing video

1. Drag in an MP4/MOV/WebM or browse for a file accepted by the media engine.
2. Dhap reads duration, frame rate, resolution, audio tracks, and basic metadata. It does not upload the file.
3. The user chooses a detection level: manual only, suggested frames, or suggested frames plus optional transcript analysis.
4. Dhap proposes candidate moments with timestamps and thumbnails. Each candidate must be accepted, edited, or removed.
5. The app creates a guide draft from accepted moments; every step retains its source timestamp.

## 6.4 Create from screenshots

1. Add multiple images or paste screenshots.
2. Reorder and group images into steps; crop and annotate each one.
3. Enter titles/instructions or use optional AI assistance.
4. Export a guide without requiring a video timeline.

## 6.5 Generate and refine steps

1. The app proposes key moments from scene-change heuristics, frame difference, pauses, optional OCR, and optionally transcript boundaries.
2. Each proposal shows a confidence hint and reason when practical (for example, “screen changed” or “long pause”). Confidence should not be displayed as a precise probability unless calibrated.
3. The user selects a frame, chooses a different frame, combines duplicates, splits a step, reorders steps, or writes a new one.
4. A step contains a title, imperative instruction, screenshot(s), optional expected result, optional notes, and source reference.
5. Generated wording is visibly marked as a draft until approved. Keep original capture facts separate from model-generated prose.

## 6.6 Annotate and redact

1. Select a screenshot or frame.
2. Add arrow, number marker, box, circle, freehand line, label, crop, blur, pixelation, or opaque redaction.
3. Redaction must be destructive in the exported image (baked into output) even if the project retains the original internally for editing. Provide a clear choice between reversible project editing and safe export rendering.
4. Before export, show a redaction checklist and allow frame-by-frame review.
5. Re-render all outputs after an annotation or redaction change.

## 6.7 Optional AI assistance

1. The user explicitly enables a local model or chooses a cloud provider and supplies their own key.
2. Before sending any data, show what will be sent: selected text, transcript, screenshots, frame descriptions, or a combination.
3. The user can redact first, send only selected steps, or cancel.
4. Model outputs come back as suggestions linked to source frame IDs and timestamps; they do not overwrite the original text silently.
5. The user can compare, accept, edit, or reject suggestions. Store provider/model metadata, but do not store secrets.
6. If the model fails or is unavailable, the rest of the app continues normally.

## 6.8 Preview, validate, and export

1. The user selects outputs: video/GIF, Markdown bundle, HTML guide/site, PDF.
2. Dhap shows a destination preview and resolves relative paths for images/assets.

3. Preflight checks find missing screenshots, empty steps, invalid links, incomplete annotations, unresolved redaction warnings, and unsupported media.
4. HTML output is previewed in an isolated local browser/webview or generated for the user to open locally.
5. Export writes to a temporary destination, verifies expected files, then atomically replaces the previous export where possible. Never leave a half-written final folder.
6. Show a simple “Open folder” / “Copy Markdown” / “Preview HTML” completion screen.

### 6.9 Reopen and update a guide

1. Open a .dhap project or recent project.
2. Dhap validates the schema version and offers migration only after creating a backup.
3. If the original video has moved, ask the user to relink it; never silently substitute a different file.
4. Changes update the guide model. The export can be regenerated without manual cleanup.
5. Optional local snapshots capture named milestones; the user can compare and restore them.

### 6.10 Batch workflow (later)

Queue multiple source recordings, choose a shared template and output folder, and review each generated guide before the batch publishes. Batch mode must never auto-share externally without explicit consent.

## 7. Functional requirements

Priority definitions: **P0** required for v1; **P1** desirable once P0 is stable; **P2** later release. The identifiers below should become GitHub issues or epics.

### 7.1 Capture and media

- **CAP-001 (P0):** Record supported display/window sources through the selected native capture foundation.
- **CAP-002 (P0):** Record a clear session start/end, duration, and source metadata.
- **CAP-003 (P0):** Pause/resume/stop without losing already written media.
- **CAP-004 (P0):** Import supported video and image formats with clear unsupported/corrupt-file errors.
- **CAP-005 (P0):** Maintain original media as immutable source; edits are nondestructive.
- **CAP-006 (P0):** Provide a timestamp marker during capture.
- **CAP-007 (P1):** Record opt-in mouse/click markers when the platform permits; never record typed text by default.
- **CAP-008 (P1):** Store webcam/microphone/audio settings and transcript alignment when supported by upstream.
- **CAP-009 (P2):** Browser extension and external recorder adapters.

### 7.2 Detection and guide model

- **GUIDE-001 (P0):** Create/edit/delete/reorder steps.
- **GUIDE-002 (P0):** Attach one or more source frames to each step.
- **GUIDE-003 (P0):** Preserve original timestamp and source ID per frame/step.
- **GUIDE-004 (P0):** Generate candidate keyframes through configurable frame sampling and scene-change heuristics.
- **GUIDE-005 (P0):** Allow manual acceptance/rejection of all generated steps.
- **GUIDE-006 (P0):** Edit title, instruction, expected result, notes, and image alt text.
- **GUIDE-007 (P1):** Group steps into titled sections and produce a table of contents.
- **GUIDE-008 (P1):** Add OCR text and transcript snippets as evidence/context.
- **GUIDE-009 (P2):** Use browser/desktop action-event adapters for more accurate action descriptions.

### 7.3 Annotation and privacy

- **ANNO-001 (P0):** Add arrows, numbers, rectangles, circles, highlights, text labels, and crop.
- **ANNO-002 (P0):** Add blur, pixelation, and opaque redaction masks.

- **ANNO-003 (P0):** Retain editable annotation metadata in the project.
- **ANNO-004 (P0):** Bake annotations and redactions into exported screenshots/media as requested.
- **ANNO-005 (P0):** Add an export checklist for manual privacy review.
- **ANNO-006 (P1):** OCR-assisted warning for apparent email addresses, access tokens, phone numbers, and password-like fields; treat as a helper, not a guarantee.
- **ANNO-007 (P2):** Automated multi-frame tracking for redaction regions.

## 7.4 AI and language assistance

- **AI-001 (P0):** Core editing and export work with AI completely disabled.
- **AI-002 (P1):** Optional provider adapter for step-description suggestions.
- **AI-003 (P1):** Optional transcription/summary integration using an available local engine.
- **AI-004 (P1):** Translate or rewrite selected instructions while preserving source references.
- **AI-005 (P1):** Return results as reviewable suggestions; never overwrite silently.
- **AI-006 (P1):** Show a pre-flight disclosure of what content would leave the device.
- **AI-007 (P2):** “Check against source” review for missing/unsupported statements, returning citations to frames/timestamps rather than a blanket guarantee of correctness.
- **AI-008 (P2):** Plugin/provider API after internal interfaces stabilize.

## 7.5 Export and publishing

- **EXP-001 (P0):** Export Markdown with local relative image paths and useful headings.
- **EXP-002 (P0):** Export clean HTML with self-contained or adjacent assets and responsive typography.
- **EXP-003 (P0):** Export printable PDF with selectable text where feasible.
- **EXP-004 (P0):** Preserve video MP4 and GIF exports already available from the base where verified.
- **EXP-005 (P0):** Provide preview, output destination choice, and preflight validation.
- **EXP-006 (P0):** Export reproducibly from the same project state.
- **EXP-007 (P1):** Templates for GitHub README, technical docs, internal SOP, tutorial, and classroom handout.
- **EXP-008 (P1):** Interactive HTML walkthrough with step navigation and media timestamps.
- **EXP-009 (P1):** Self-contained documentation site folder ready for static hosting.
- **EXP-010 (P2):** Direct push/publish integration to a provider or Git repository, requiring explicit user action and scoped credentials.

## 7.6 Project and workspace

- **PRJ-001 (P0):** Create, save, open, rename, duplicate, and delete projects.
- **PRJ-002 (P0):** Autosave with visible save state and crash-safe writes.
- **PRJ-003 (P0):** Versioned .dhap bundle with assets and schema migration support.
- **PRJ-004 (P0):** Recent-project list; local library index is optional if it complicates initial delivery.
- **PRJ-005 (P1):** Named local snapshots, restore, and project comparison.
- **PRJ-006 (P1):** Reusable style/export presets.
- **PRJ-007 (P2):** Batch processing and queue management.
- **PRJ-008 (P2):** Shared projects and real-time collaboration.

## 7.7 Automation and integrations

- **AUTO-001 (P1):** Stable CLI operations for inspect, import, validate, export, and batch generation.
- **AUTO-002 (P1):** Machine-readable JSON/NDJSON result format and stable exit codes.
- **AUTO-003 (P1):** MCP tools for safe, explicit project operations after CLI contracts stabilize.
- **AUTO-004 (P2):** Browser extension uses the shared project model, not a separate guide format.
- **AUTO-005 (P2):** Plugin system with capability declarations and permissions.
- **AUTO-006 (P2):** GitHub Actions examples for regenerating docs from checked-in sources; no secret should be required for local exports.

# 8. Information architecture and UI/UX

## 8.1 Main navigation

- **Home / Recent:** create a project, import recording, recover recent work, view sample project.
- **Capture:** source selection, audio options, privacy indicators, capture markers.
- **Editor:** switchable Document, Timeline, and Preview layouts.
- **Guide:** ordered step list, sections, step text, source links, image annotations.
- **Media:** sources, clips, frame library, transcripts, generated thumbnails.
- **Export:** output presets, path, format options, safety checklist, export history.
- **Settings:** appearance, shortcuts, directories, media cache, AI providers, updates, privacy, accessibility.

## 8.2 Flexible workspace model

Do not create three separate editors with different behavior. Keep one underlying project state and offer layout presets:

1. **Guide layout (default):** step list and text editor as the main canvas, with screenshot preview and source timestamp. Best for documentation work.
2. **Timeline layout:** video timeline and source preview are prominent; the guide-step lane is linked to timestamps. Best for editing a recording into a guide.
3. **Preview layout:** rendered output takes most of the space; document outline and output format controls remain visible. Best for final review.

The user can resize panels, collapse secondary areas, and reset the layout. Remember layouts per project or per workspace, but avoid making layout persistence a launch blocker.

## 8.3 Key screens

1. Welcome/home with three primary actions and one sample project.
2. Capture-source chooser with permissions and device readiness.
3. Recording HUD with unmistakable recording state, pause, mark-step, and stop controls.
4. Import/progress view with cancellation and meaningful failure recovery.
5. Editor workspace with source preview, guide steps, and optional timeline.
6. Annotation canvas with straightforward tools and undo/redo.
7. AI suggestion review panel with before/after comparison and provider disclosure.
8. Export settings and validation checklist.
9. Export preview and success state with quick actions.
10. Settings for theme, shortcuts, privacy, media cache, model/provider, updates, and diagnostics.
11. Project recovery/migration dialog with backup-before-change policy.

## 8.4 UX rules

- Every long-running process shows progress, the current phase, and a safe cancel option.
- Errors explain what happened and how to recover. “Something went wrong” alone is not an acceptable user-facing error.
- Never lose unsaved work because an export or AI request failed.
- Use undo/redo for destructive-looking editing actions where feasible.
- Keep the primary action visually clear; don’t use multiple high-saturation colors as decoration.
- Do not force a tutorial, sign-in, API-key setup, telemetry consent, or subscription choice before first value.
- Keyboard shortcuts are documented, visible in menus, and remappable only if the cost is justified.
- Provide tooltips without hiding core meaning in tooltips alone.
- Respect screen scaling, high DPI, reduced motion, OS theme, and keyboard-only navigation.
- Keep window geometry and panel state sensible on 1366×768 screens as well as large displays.

### 8.5 Suggested visual system

- Typography: one highly legible UI family plus an excellent reading face for guide preview; include local fallback fonts and do not depend on remote fonts at runtime.
- Spacing: 4/8-point rhythm, compact but not cramped; generous reading width in document mode.
- Surfaces: neutral, layered panels; reserve paper-like texture or hand-drawn marks for guide output and onboarding rather than putting texture behind every control.
- Color: light mode uses a warm neutral background and ink text; dark mode uses deep slate rather than absolute black. Recording/error states require distinct semantic colors and accessible contrast.
- Illustrations: original small diagrams demonstrating a recording becoming a guide. Use consistent stroke widths and editable vectors.
- Redaction: use a strong warning color with clear preview state; don't rely on color alone to indicate redacted content.
- Exports: prioritize legibility and print quality over app-like effects. Every template must work without remote assets.

## 9. Recommended technology and architecture

### 9.1 Decision

**Recommended: fork/adapt OpenScreen for the first generation, with a Rust media/core layer and its existing Electron + TypeScript/React application shell.** Do not migrate the entire application to Tauri or rewrite capture from scratch in v1.

The user preference is Rust for performance, but the fastest maintainable solution is not "Rust everywhere." Recording, video processing, frame extraction, rendering, and bundle operations benefit from native code; a flexible multi-panel editor is efficient to develop in TypeScript/React. OpenScreen already combines native capture APIs and a Rust GPU rendering core with a cross-platform app. [1]

### 9.2 Why not a fresh Tauri application?

A greenfield Tauri/Rust UI could yield a smaller shell, but would still require screen capture integrations, audio/video synchronization, media import, rendering parity, installers, OS permission handling, and testing. Porting or duplicating those hard parts consumes the entire initial timeline. Consider a Tauri migration only after Dhap has a proven product and a measured reason to move.

### 9.3 Proposed layers

1. **Desktop shell:** existing Electron app, menus, window management, OS permission flows, file dialogs, update handling.
2. **UI:** TypeScript + React, typed state/store, component system, accessible keyboard model, switchable workspace layouts.
3. **Media/capture layer:** inherited OpenScreen capture and rendering paths; FFmpeg or existing media tooling for supported transcodes and image sequence extraction. Do not add a second media pipeline before benchmarking.
4. **Dhap guide domain:** a separate, versioned project/step model, tested independently from the UI.
5. **Frame analysis:** deterministic sampling/scene-change heuristics, pluggable OCR/transcription inputs, timestamps retained throughout.
6. **Annotation/rendering:** vectors and redaction metadata stored nondestructively; output renderer bakes the chosen results into screenshots/video only when exporting.
7. **Export layer:** Markdown, HTML/site folder, PDF/print output, plus inherited MP4/GIF exports. Each exporter consumes the same guide model.
8. **AI provider layer:** optional, explicit, replaceable adapter. No provider logic should own the project model.
9. **Local persistence:** versioned .dhap archive and atomic write/recovery helpers; lightweight local database only for search/indexing if necessary.
10. **Automation API:** CLI/MCP call the same application/domain operations as the UI, not private UI selectors.

## 9.4 Integration boundary with OpenScreen

The public OpenScreen docs warn that .openscreen and CLI interfaces can change between releases. [1] Therefore:

- Pin the initial upstream base to a tested commit/release.
- Record the base commit and upstream attribution in UPSTREAM.md.
- Keep upstream-specific APIs behind an adapter module.
- Avoid making .openscreen the canonical Dhap guide file.
- Add golden fixtures for imported upstream projects and CI tests for every adapter change.
- Maintain an upstream-tracking branch and a documented merge cadence.
- Never assume an upstream private function is a stable extension point without a pinned dependency/contract test.
- If integration becomes too invasive, keep Dhap a separate application that imports common media/project assets through a documented format; do not let the architecture indecision block the workflow prototype.

## 9.5 Stack selection checklist

| Area | Initial choice | Reason / validation needed |
|---|---|---|
| Desktop shell | Existing Electron shell from evaluated foundation | Avoid rewriting capture, permissions, and packaging |
| UI | TypeScript + React | Fast iteration for complex editor layout and export preview |
| Performance-critical core | Rust | Reuse native rendering, safe media/project operations, common core behavior |
| GPU rendering | Existing upstream renderer | Preserve preview/export parity; benchmark before modifying |
| Media | Existing upstream pipeline + FFmpeg/tooling already in use | Avoid parallel codec stacks; validate licences and platform behavior |
| Guide schema | Versioned JSON inside a .dhap bundle | Portable, inspectable, and migratable |
| Local library index | Start with filesystem + recent-project metadata; add SQLite only if query needs justify it | Avoid unnecessary storage complexity |
| Tests | Rust unit/integration tests, TypeScript tests, Playwright/Electron E2E where supported | Test domain separately from OS-specific capture |
| Static site | Static site deployed to Cloudflare Pages through GitHub Actions | Matches the owner's current release/deployment strengths |
| Optional hosting/API | No server required for v1; Cloudflare Worker only for a later opt-in hosted feature | Keeps privacy, operational cost, and security scope small |

## 9.6 Repository layout (proposed)

```
Dhap/
  apps/desktop/      # Electron + React UI
  apps/site/         # website and documentation
  crates/project/    # versioned schema, validation, migrations
  crates/analysis/   # frame sampling and step suggestions
```

```
 crates/annotations/ # annotation geometry and redaction
 crates/export/      # Markdown, HTML, PDF orchestration
 crates/cli/         # stable command-line interface
 packages/ui/        # shared components and design tokens
 packages/schema/    # generated TypeScript schema bindings
 fixtures/           # synthetic media and sample projects
 tests/e2e/          # UI and packaged smoke tests
 tests/golden/       # expected exports and image fixtures
 packaging/          # installers, signing, portable artifacts
 docs/               # architecture, privacy, troubleshooting
 .github/workflows/  # CI, E2E, deploy, release
 UPSTREAM.md
 THIRD_PARTY_NOTICES.md
 CONTRIBUTING.md
 SECURITY.md
```

This is a logical layout, not a mandate to create every folder on day one. If the actual upstream monorepo conventions make this structure disruptive, adapt the names while maintaining equivalent boundaries.

# 10. Project format and data model

## 10.1 .dhap bundle

Use .dhap as a versioned, portable project package. A bundle can be implemented as a ZIP-compatible archive with a manifest and relative asset paths; test corruption recovery before promising format-level stability.

```
my-guide.dhap
 manifest.json
 project.json
 media/
  source-001.mp4          # optional embedded source, if user chooses portability
 frames/
  frame-0001.webp
 annotations/
  annotation-0001.json    # optional separation if useful
 transcripts/
  transcript-001.json
 thumbnails/
 exports/                 # optional; may be excluded to keep the bundle small
```

Projects should support two media modes: **linked source** for smaller project metadata and **portable bundle** where the user elects to include source media. Always make the mode obvious. Never silently duplicate huge source files or silently break links. Each bundle contains a schema version, app version, stable IDs, relative paths, and checksums for critical assets.

## 10.2 Core entities

- **Project:** ID, title, created/modified timestamps, schema version, export defaults, privacy settings, source list, ordered sections/steps.

- **SourceMedia:** ID, file reference, media type, duration, frame rate, resolution, audio tracks, content hash if practical, linked/embedded status.
- **CaptureSession:** ID, start/end, capture configuration, source IDs, optional recorded event markers, permission state metadata without sensitive system details.
- **Frame:** ID, source media ID, timestamp in milliseconds, thumbnail path, optional OCR/transcript references, selection origin (manual/detection/import), detection reason.
- **GuideSection:** ID, title, ordering, step IDs.
- **GuideStep:** ID, title, instructions, expected result, notes, alt text, frame IDs, section ID, ordering, source timestamp references, review status.
- **Annotation:** ID, target frame/clip, type, geometry, style, redaction flag, output-rendering behavior.
- **TranscriptSegment:** ID, source ID, start/end milliseconds, text, optional speaker label, origin and confidence metadata.
- **AISuggestion:** ID, target field/step, proposed text, provider/model, source evidence IDs, created timestamp, status (pending/accepted/rejected), never contains API secrets.
- **ExportProfile:** format, template, page/video options, output directory, asset strategy, last export fingerprint.
- **ProjectSnapshot:** snapshot ID, timestamp, label, schema version, reference to snapshot content or diff.

### 10.3 Data integrity rules

- Use UUID/ULID-like stable identifiers, not array positions, to refer to steps and frames.
- Use integer milliseconds or a documented rational time base for timestamps; avoid float-based drift.
- Store assets using relative paths; reject path traversal on import and extraction.
- Save with write-to-temp, flush, verify, then atomic rename where the platform permits.
- Keep a backup before migration; if migration fails, preserve the untouched original.
- Add schema migrations as explicit, tested functions. Keep fixtures from older versions and test opening them in every release.
- Projects should remain readable if an optional AI transcript/metadata file is missing.
- No provider key, token, or private credential belongs in project.json, logs, exported guides, or crash reports.

## 11. Privacy, security, and responsible AI

### 11.1 Default data flow

The base workflow is local: capture → local project → guide edit → local export. There is no application backend requirement. Network access may occur only for an explicitly configured AI provider, optional model download, user-requested publishing, or update checks governed by settings. Explain each case clearly.

### 11.2 AI privacy controls

- No AI provider is called before the user configures and chooses it.
- Show exactly what will be sent and which provider will receive it.
- Offer “selected step only” and “redact before sending” controls.
- Store keys in OS secure credential storage, not project files or normal settings JSON.
- If a provider is used, explain that its own retention/training policy applies; do not promise the provider will store nothing unless verified.
- Never send an entire source recording by default when text or selected frames are sufficient.
- Make AI output visibly different from verified source facts, with one-click reject/undo.
- The app must remain functional without credentials and after provider failure.

### 11.3 Redaction safety

Redaction is safety-critical because screen recordings often contain email addresses, tokens, names, addresses, or private records. The application must:

- Support opaque solid masks as the most reliable manual redaction method; blur alone may not fully conceal information.

- Bake redactions into every affected output when “safe export” is selected.
- Ensure the original unredacted screenshot is not accidentally copied into the output bundle when the user selected a redacted-only package.
- Render previews from the same export pipeline used for final output wherever feasible.
- Include a manual review checklist and a clear statement that automated detection cannot guarantee privacy.
- Add regression tests that inspect exported asset pixels and scan archive contents for expected omissions.

### 11.4 Security controls

- Validate imported archives and relative paths; guard against ZIP path traversal and decompression bombs.
- Avoid executing imported project code, templates, or plugin code automatically.
- Sanitize Markdown/HTML and avoid unsafe HTML injection from AI/user content.
- Use restrictive Content Security Policy for website and exported viewer where compatible.
- Treat external links and remote images as external; do not fetch them implicitly during export or preview without consent.
- Pin critical GitHub Actions to immutable commit SHAs where possible; use least-privilege permissions per workflow.
- Add dependency/license inventory, vulnerability scanning, secret scanning, SBOM generation, and signed/checksummed release artifacts as the project matures.
- Define a security contact and responsible disclosure policy before public launch.

## 12. CI/CD and release automation

### 12.1 Learn from the public Khata workflows

I inspected the public Khata repository, its documented hosting setup, CI workflow, release workflow, and site deployment workflow. The public implementation demonstrates a useful “set up once, then automate” pattern: cargo check and selected tests, dependency licence audit, site checks, a one-click or tag-triggered versioned release, per-platform packaging, checksums and release notes, a separate Cloudflare Pages landing-site deployment, and a separate GitHub Pages web-app deploy because the WebAssembly file exceeds Cloudflare Pages’ 25 MiB per-file limit. The landing-site deployment checks files/links and performs a post-deploy smoke check. [14][15][16][17]

The inspected public ci.yml currently runs its core check job on Ubuntu; the release workflow builds Windows/Linux as required and macOS/web as best-effort. I did **not** see a full cross-OS GUI E2E matrix in that CI file. Dhap’s automation should keep the simple one-click release model but explicitly add the cross-platform and packaged-app E2E coverage requested here, rather than assume it already exists.

### 12.2 Workflow inventory

| Workflow | Trigger | Required behavior |
|---|---|---|
| ci.yml | Pull request, push to main, manual | Formatting, lint, type checking, Rust checks/tests, JS tests, schema tests, license policy, security scanning |
| e2e.yml | PR smoke tests; nightly full matrix; release candidate | Desktop UI flows, record/import fixtures, guide creation, annotation/redaction, exports, reopen/migrate project |
| build-matrix.yml | PR or main for changed areas; nightly as needed | Build supported OS/architecture targets, upload test artifacts, publish readable test summaries |
| site.yml | PR and main when site changes | Link/assets/security headers checks, accessibility/HTML validation, link checking, site build |

| Workflow | Trigger | Required behavior |
|---|---|---|
| deploy-site.yml | Main after successful checks, manual dispatch | Deploy marketing/docs site to Cloudflare Pages and smoke test the live deployment |
| release.yml | Tag v* or manual dispatch | Dry-run support, version validation, build installers/portable archives, checksums, release notes, GitHub Release |
| post-release.yml | Successful release | Verify release assets/checksums, download and install smoke tests where feasible, publish update metadata and launch docs |
| dependency-review.yml | Pull request and schedule | Dependency review, advisories, license allow/deny policy, SBOM and vulnerability reports |

Workflow names may be adjusted to match the eventual repo. Prefer a few reusable workflows over many duplicated YAML files.

## 12.3 CI quality gates

**Every pull request:** - Rust formatting and clippy (or upstream equivalents). - TypeScript/React lint and type checking. - Rust domain tests and UI/component tests. - Schema serialization/deserialization round trips. - Old-project migration fixtures. - Export golden tests for Markdown/HTML and image rendering. - License audit and security/dependency review. - Build smoke on Linux; selected Windows/macOS checks when relevant.

**Nightly / scheduled matrix:** - Windows, macOS, and Linux package builds on supported GitHub-hosted runners. - Full E2E suite with deterministic sample media. - Performance baselines for frame extraction and export. - Accessibility checks on key web/docs UI. - Dependency vulnerability scan and stale issue/PR checks only if useful; avoid noisy automation.

**Release candidate/tag:** - Build exact commit/version on each target OS. - Run smoke tests against packaged artifacts, not only development builds. - Verify a sample project opens and exports a guide and video. - Verify portable builds in a clean environment where feasible. - Generate SHA-256 checksums and an SBOM/third-party notices. - Publish a draft release, then promote to public only if required checks pass.

## 12.4 E2E test matrix

Use a layered test strategy: unit tests for domain logic; integration tests for media and exports; Playwright/Electron UI tests for workflows; packaged smoke tests for release artifacts; and limited physical/self-hosted capture tests where hosted runners cannot reliably validate OS capture permissions or audio devices.

| Area | Linux | Windows | macOS |
|---|---|---|---|
| Build / unit tests | Every PR | PR or main matrix | PR or main matrix |
| Guide-editor E2E using fixtures | Every PR if runtime allows | Every PR if runtime allows | Main/nightly if expensive |
| Packaged install + launch | Release candidate | Release candidate | Release candidate |
| Screen-capture permission/device test | Scheduled controlled runner or mocked capture | Scheduled controlled runner or mocked capture | Scheduled controlled runner or mocked capture |
| Export output comparison | Golden fixtures | Golden fixtures | Golden fixtures |
| Older supported OS coverage | Supported LTS container/VM where feasible | Explicit supported runner/VM where feasible | Supported runner/VM where feasible |

Do not force every expensive GUI test into every pull request if it makes iteration slow. Keep a fast PR gate, a complete nightly suite, and a mandatory release-candidate validation. Every skipped test should be reported in the workflow summary rather than silently ignored.

## 12.5 One-click release behavior

The release workflow should mirror the best aspects of Khata's current public release pattern:

1. Manually enter a semantic version or trigger by a v* tag.
2. Validate the version and ensure the working tree/tag is consistent.
3. Support a dry run that builds and tests without committing a version bump or publishing artifacts.
4. Build in parallel; make required platform policy explicit.
5. Generate third-party licence notices, release notes, checksums, and an SBOM.
6. Create a draft GitHub Release with all files and test summaries.
7. Verify downloads/checksums and a smoke launch.
8. Promote the release, then deploy the website/download metadata.
9. On failure, preserve artifacts and logs for diagnosis; never publish a partial release as stable.

## 12.6 Hosting and Cloudflare policy

- Host the static marketing site, documentation, and launch pages on Cloudflare Pages through a scoped GitHub Actions token. Preflight checks should run before deployment; post-deploy smoke tests should confirm the deployment serves the expected content.
- No Worker is required for the local-first v1. Add a Cloudflare Worker only if a concrete optional service justifies it, such as opt-in hosted guide publishing or short links. Keep that service independent of local recording/editing.
- Use a separate build/deploy target if the static guide viewer or large WebAssembly bundle exceeds hosting limits. Document the choice and do not hide storage/transfer limits from maintainers.
- Protect secrets in GitHub repository/environment settings, scope tokens to the single project, and ensure PRs from forks cannot access deploy secrets.
- Every hosting workflow should have manual dispatch for recovery and a concise summary containing the environment, commit SHA, and deployed URL.

# 13. Testing and acceptance criteria

## 13.1 Release-blocking user acceptance tests

1. A new user can launch the app, import a short video, create five steps, and export a Markdown + HTML guide without signing in or configuring AI.
2. The guide's screenshots map to correct timestamps and remain linked after reordering steps.
3. A redacted value is obscured in every exported format selected by the user, and an unredacted source asset is not included in a redacted-only export package.
4. The project can be closed and reopened with all step text, frames, annotations, timeline references, and export settings intact.
5. An interrupted export does not corrupt the project or destroy a previously successful output.
6. Unsupported or corrupt media produces a clear message and leaves the original file untouched.
7. A .dhap bundle from a previous supported schema version opens through an automated migration test with a backup.
8. AI is disabled by default or until explicit setup, and core flows pass with no network connection.
9. Provider failure leaves the app stable; generated text is only applied after user acceptance.
10. Keyboard-only users can perform the critical import/edit/export flow, with visible focus and an accessible status announcement.
11. Packaged installers start on all release platforms and can open/export the sample project.
12. The release workflow can run as a dry run with no public release or version commit.

## 13.2 Performance goals (targets to validate, not unsupported guarantees)

- App reaches an interactive home screen within 5 seconds on a typical modern laptop after first launch, excluding OS-specific setup or first-time model downloads.
- Opening a project does not decode the full source video into memory; thumbnails and frames are lazy-loaded.
- Scrubbing remains responsive by using generated proxies/thumbnails and asynchronous background extraction.
- Autosave work must not block the editor for noticeable periods; large media copying occurs in background with progress.
- Cancellation stops a long-running export or analysis promptly where the underlying library supports cancellation.
- The app provides progress and remaining-stage information instead of appearing frozen.
- Maintain an automated benchmark for frame sampling, project open/save, and export; set hard budgets only after baselines are measured across systems.

## 13.3 Reliability targets

- No known data-loss bug is acceptable for a stable release.
- Autosave, crash-recovery, and project-migration features have dedicated fault-injection tests.
- Exporters must have golden fixtures checked for broken paths, missing images, or malformed HTML.
- Redaction tests inspect the rendered pixel output and packaged contents, not only UI state.
- Every crash report and support bundle is opt-in and must exclude source media, screenshots, project text, API keys, and private file paths unless the user explicitly chooses to include specific logs.

# 14. Roadmap: 12-week solo execution plan

## Weeks 1–2: validate and de-risk

- Confirm a small user cohort (8–12 people) and collect real example workflows/recordings with permission.
- Benchmark a short OpenScreen fork spike: record/import media, select frames, add a step, render Markdown/HTML.
- Inspect project license, all relevant dependencies, build targets, media pipeline and upstream extension points.
- Create product logo direction, 2–3 editor prototypes, and test them with 3–5 users before detailed polish.
- Decide whether the docs workflow is a module inside the fork or a sibling app that shares a stable adapter. Make the decision from code access and change size, not aesthetic preference alone.

**Gate:** if adding a guide editor forces invasive changes across the recording/rendering core, switch to a separate app that imports media and does not depend on private OpenScreen internals.

### Weeks 3–4: project model and basic editor

- Implement versioned project model and .dhap save/open.
- Build frame selection, manual step list, title/instruction editing, reorder, and source timestamps.
- Create basic screenshot output and Markdown exporter.
- Add autosave, crash-safe write, and a sample project.
- Start unit, schema, and export golden tests immediately.

### Weeks 5–6: annotations and HTML/PDF output

- Add arrows/number markers/highlights/crop and redaction.
- Generate styled HTML guide with local assets and responsive behavior.
- Generate PDF through a reliable print/render pipeline.
- Add output preview, validation, and recovery for failed exports.
- Keep exported assets self-contained or explain linked assets clearly.

### Weeks 7–8: detection and polish

- Add configurable frame sampling and scene-change-based candidate steps.
- Wire optional transcript/AI provider behind a privacy preflight, only after manual editing/export is reliable.
- Add light/dark themes and layout switching.
- Conduct usability checks on small and large screens; fix the top friction points.
- Add tests for redaction, archive paths, old schema migrations, and interruption recovery.

### Weeks 9–10: automation and packaging

- Cross-platform build matrix; packaging on each platform; portable artifacts where feasible.
- Fast PR CI, nightly full E2E, release-candidate checks, dependency/licence audit.
- Cloudflare Pages website deploy and post-deploy smoke test.
- One-click semantic release, dry-run mode, release notes/checksums, third-party notices.
- Test supported OS versions and clean installs; document remaining signing requirements.

### Weeks 11–12: public launch quality

- Close release-blocking bugs and accessibility issues.
- Produce a short original product demo and before/after workflow examples.
- Complete website, docs, FAQ, privacy model, troubleshooting, security policy, contribution guide, and sample project.
- Publish a draft release to testers; verify installers, exports, recovery, and checksums.
- Launch the stable version with transparent known limitations and a simple feedback channel.

**Scope rule:** if the project slips, defer browser extensions, full web editor, collaboration, complex batch processing, and elaborate plugin APIs. Do not defer export reliability, privacy review, project integrity, installability, or a coherent first-run experience.

# 15. Public release, website, and community

## 15.1 Website information architecture

- Home: one-sentence value proposition and an actual animated or recorded workflow demo.
- Features: screen recording, guide editing, annotations/redaction, export formats, local-first privacy.
- Gallery: real sample guides in Markdown, HTML, and PDF.

- Download: installers, portable packages, supported OS requirements, signatures/checksums, release notes.
- Documentation: quick start, recording workflow, editing, exports, privacy/redaction, AI setup, troubleshooting, CLI.
- Project: roadmap, changelog, upstream attribution, license, contributors.
- Community: GitHub issues/discussions, contribution guidelines, security contact.

### 15.2 GitHub repository readiness

- A useful one-sentence About description and accurate topics.
- README with a clear demo GIF/video, quick install, actual screenshots, feature status badges, and a concise “why not just use a recorder?” section.
- Changelog and versioned releases with checksums and notices.
- Contribution guide explaining architecture, local setup, test commands, and design principles.
- CODE_OF_CONDUCT.md, SECURITY.md, issue forms, pull-request template, and roadmap.
- First good issues and small scoped issues to welcome contributors.
- Explicit status labels: stable, experimental, planned. Do not present roadmap ideas as shipped features.

### 15.3 Launch sequence

1. Private prototype review with the first user cohort.
2. Public alpha with a small number of critical workflows and a feedback form.
3. Publish a 30–60 second demo: record a task → generate a guide → edit/redact → export video and Markdown/HTML.
4. Release a polished beta with real sample project and cross-platform package smoke tests.
5. Launch v1.0 on GitHub, the product website, relevant open-source communities, and developer/creator forums with honest scope and limitations.
6. Convert repeated user requests into public roadmap items and short release notes.

Avoid spammy self-promotion. Show the complete workflow and ask for concrete feedback from people who create documentation regularly.

## 16. Sustainability and commercial strategy

Dhap’s local application should remain genuinely useful for free. No watermarks, export caps, account wall, or surprise removal of features. Optional future revenue can support development without undermining the promise.

Potential later revenue: - Optional hosted publishing and managed links for users who choose to use a hosted service. - Team collaboration and shared template libraries as separate convenience services. - Business support, onboarding, custom template/design work, and sponsorship. - Donations, GitHub Sponsors, or community funding.

Keep self-hosted/static export possible and document the export format. Avoid using collected content as a business asset. Do not introduce subscriptions until there is demonstrated demand and sustainable service cost. Core local editing/export functionality must continue to work if a paid service closes.

## 17. Licensing and upstream responsibilities

OpenScreen’s published project is MIT-licensed, allowing copying, modification, distribution, and commercial use subject to the licence’s conditions. [1][2] The final choice must be confirmed against the exact upstream commit and every dependency actually shipped.

Recommended policy: - If forking OpenScreen, preserve upstream copyright/license notices and include full MIT license text and third-party notices as required. - Keep a clear file such as UPSTREAM.md explaining the

forked project, base commit, changes, and upstream links. - Use the Dhap name, icon, and website independently; do not imply the project is official OpenScreen or reuse marks in a confusing way. - Run an automated licence inventory/allowlist in CI, following the approach already used by Khata. - Prefer MIT for Dhap-specific code if all dependencies and project goals support it; if a future dependency requires another compatible licence, resolve this before merging it rather than blindly promising MIT. - Keep AI model licenses and model weight redistribution terms separate from code-license checks. - Get legal review if Dhap is commercialized or if its relationship to the upstream project becomes complicated.

# 18. Risks and mitigations

| Risk | Why it matters | Mitigation |
|---|---|---|
| Scope explosion | The full requested feature set exceeds a 12-week solo schedule | P0/P1/P2 scope, user-validation gates, defer full web editor/collaboration/extension |
| Upstream API churn | OpenScreen's CLI/project format can change | Pin base, adapter boundary, fixture tests, scheduled upstream review |
| Weak differentiation | Mimik and OpenSteps already generate step guides | Focus on unified desktop recording + video + multi-format documentation from one project |
| Incorrect step generation | Video-only scene detection can miss task semantics | Keep all suggestions editable; manual markers; show evidence/source timestamps |
| Privacy leak | Videos can contain secrets/private data | Local-first, optional AI, redaction preflight, baked output, tests inspect packages |
| Cross-platform capture differences | Screen/audio APIs and permissions vary by OS | Use upstream capture; E2E fixtures; scheduled device tests; document known platform gaps |
| Export instability | Broken relative assets make a guide unusable | Golden tests, staged export, post-export verification, self-contained HTML presets |
| Licensing mistakes | Forks and media dependencies carry obligations | Automated licence inventory, attribution, model-license review |
| Domain/trademark conflict | Short names are often already used | Complete domain and mark screening before public brand launch |
| Update frustration | Forced updates disrupt work and undermine trust | Background download optional; notify user; no restart or forced close during active work |
| Electron size/performance | Installers can be large, startup can regress | Reuse proven stack, lazy-load, benchmark, remove unused dependencies only when measured |
| Unsigned installers | OS security warnings can deter users | Add Windows/macOS signing when viable; publish checksums and clear official download path |

# 19. Decision log and unresolved items

## Decisions recommended now

- Brand candidate: **Dhap**, pending domain and trademark checks.
- Product promise: **one project produces both polished media and editable documentation**.

- Architecture: evaluate a pinned OpenScreen base and implement the guide model as a separate module; avoid a greenfield capture stack.
- Performance stack: Rust for core/media and project logic where it fits; TypeScript/React in the existing shell for a fast, accessible editor.
- Privacy: local-first; AI optional; no account needed for core use.
- Initial platform: desktop Windows/macOS/Linux via the selected base. Web v1 is an output/viewer, not the whole editor.
- Initial license direction: preserve MIT upstream requirements and prefer MIT for Dhap code if dependency audit permits.
- Release philosophy: automated multi-platform packages, one-click/dry-run workflow, cross-platform CI/E2E, checksums, generated release notes, deployment smoke tests.

### Must be settled after the first technical spike

1. Which exact OpenScreen commit/release is the best foundation, and which extension points are stable enough?
2. Whether the docs editor should live directly inside the fork or as a sibling application that imports source media.
3. PDF renderer choice and font embedding behavior across OSes.
4. Which OCR/transcription implementation offers acceptable quality, packaging size, and compatible licensing for the target languages.
5. .dhap archive strategy: ZIP-based bundle, linked media, or both; project recovery behavior.
6. Which AI provider adapters are feasible for v1 without adding a dependency on any paid API.
7. Exact supported OS versions and architectures for the first stable release.
8. Whether Windows/macOS signing certificates are available for the launch and the UX used if not.
9. Final name/domain/trademark clearance.
10. Which optional hosted service, if any, is worth building after user demand is proven.

Do not block the prototype while waiting for every answer. Run the two-week technical spike and user validation to resolve the first few architecture questions with evidence.

## 20. Definition of done for v1.0

Dhap v1.0 is ready when:

- A new user can record or import media, create a guide, annotate and redact images, and export Markdown/HTML/PDF without an account or API key.
- The same project can also produce a polished video/GIF through the retained media engine where supported.
- A project can be saved, closed, reopened, and migrated without losing guide structure or references.
- Every generated step is editable and linked to source evidence; no AI text is silently treated as verified truth.
- The app works offline for core workflows.
- Supported Windows, macOS, and Linux builds pass required packaged smoke tests; known exceptions are explicit.
- CI runs formatting, type checking, tests, license/security checks, and export regression fixtures.
- E2E coverage runs on multiple supported OS targets on a documented schedule and as a release gate.
- Release automation supports dry-run and a reproducible one-click/tag release, including checksums, generated notes, notices, and test summaries.
- The website, documentation, privacy statement, attribution, contribution guide, and security contact are published.
- The top observed usability problems from the alpha cohort are resolved or documented transparently.

## 21. Research references

Research conducted 22 October 2026. Public repository descriptions and search results can change; verify again before making final legal, release, or architecture decisions.

1. OpenScreen repository, MIT licence, codebase, recent activity and releases: https://github.com/getopenscreen/openscreen
2. OpenScreen documentation (version 2.1.0, current capture/rendering/export behavior and project-format/CLI caveat): https://getopenscreen.com/docs/intro/

3. Bengali-English dictionary entry for ধাপ: https://www.english-bangla.com/bntoen/index/%E0%A6%A7%E0%A6%BE%E0%A6%AA and Bangladesh government's searchable dictionary: https://accessibledictionary.gov.bd/bengali-to-english/?alp=&page=334
4. Noksha design/art-generation app listing: https://appshunter.io/ios/app/noksha/id6450398152
5. SOPAN active education/workflow software: https://www.sopan365.com/
6. Kadam app listings: https://apps.apple.com/in/app/kadam/id6775864938 and https://apps.apple.com/us/app/kadam/id6787451357
7. Lekha business management software: https://www.lekha.app/ and a separate ledger app reference: https://github.com/geeekyvishal/lekha
8. Poth ebook/content app listing: https://apps.apple.com/us/app/poth/id1435760792
9. Chitro video editor app listing: https://apps.apple.com/us/app/chitro-video-editor-maker/id1558665891
10. ICANN domain registration lookup: https://lookup.icann.org/en/lookup
11. WIPO Global Brand Database and WIPO trademark research guidance: https://www.wipo.int/en/web/global-brand-database/index
12. Mimik open-source browser workflow/documentation recorder, MIT: https://github.com/westpoint-io/mimik
13. OpenSteps open-source Windows desktop workflow recorder, MIT, early-beta status described in repository: https://github.com/ebanez8/openstep
14. Khata public repository and product description: https://github.com/nhprince/khata
15. Khata public CI workflow: https://raw.githubusercontent.com/nhprince/khata/main/.github/workflows/ci.yml
16. Khata public release automation: https://raw.githubusercontent.com/nhprince/khata/main/.github/workflows/release.yml
17. Khata public hosting notes and workflows: https://raw.githubusercontent.com/nhprince/khata/main/docs/hosting.md and https://raw.githubusercontent.com/nhprince/khata/main/.github/workflows/site-landing.yml
18. Bangladesh Patent and Trademark Office information portal: https://www.bdpto.org/

**Final recommendation:** build Dhap as a documentation-first extension of a proven cross-platform recorder, with one stable project model, a beautiful guide editor, trusted exports, optional AI, and automation that makes every release repeatable. Protect the project's scope ruthlessly: a reliable workflow people love is more valuable than a long feature checklist that frustrates them.